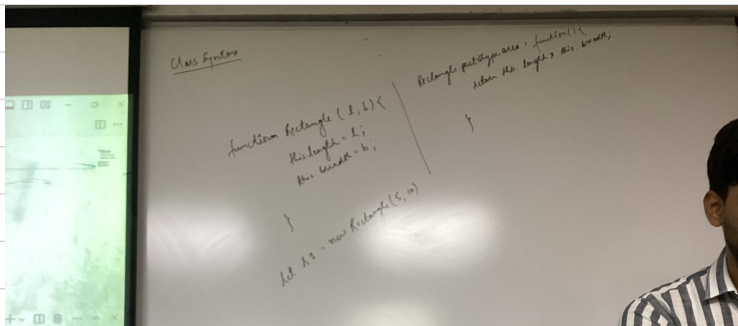



27 July, 2024

Class Syntax :-

Sir ne prototype wale code ko class syntax ka use karke likha.



Class syntax mei hum constructor function banate hai.
↳ check code

Class syntax ^{ka code} ander hi ander prototype hi use kar raha hai. ^

Class syntax was introduced to make JS similar to Java & C++.

Static function ⇒ they can be called only by the function where static function exists. Agar koi bahar ka instance call karega, then error aayega.

↓
outer function

↓
inner function

jab function ke aage "set" lgaoge, it behaves as property, not function. Otherwise error aayega.

Hamne "get" bhi pada.

check
code

Inheritance :-

Sir ne bola ki → Cuboid class ke
under class Rectangle ke parts aa jayenge

↳ check script2.js

This is why you are able to ~~access~~ use
area()

cl. ~~area~~ when cl belongs to

cuboid not Rectangle class.

Ways to write Async. code in sync manner.

① Callback use karke

Nested call back

S1 S2 S3 → 8 sec (swimmer)

S3 with S2 and S2 with S1

↪
S3 holded hand of S2, S2 holded hand of S1.

20minto food (30 min)	swifty drink (10 min)	zepto 20 min sweets
--------------------------	--------------------------	---------------------------

1 food ③
2 drinks ①
3 sweets ②

↳ iss prnce mei ~~is~~ o/p aage.

wrapper function banana padega.

call back hell or pyramid of doom → poor code readability

isliye ye wala method
jyada use nahi
karste

Sir ka code check karo → download
compress
upload
wala

⑨ Promise

Math.ceil(num);
Math.floor(num);
Math.round(2.5);
Math.random(1);

↳ 0 - 1 → my
value
b/w this.

promise
async await, DOM

Ways to write asynchronous code in synchronous manner:-

① Method 1 using ↴

File name :- Class 18
Homework, call back hell

Callbacks + callback Hell

⇒ quick revision of HOF & callback function:-

HOF is a function that either takes one or more functions as arguments, returns a function as a result or both.

```
function callbackFunction(){  
    console.log('I am a callback function');  
}  
  
function higherOrderFunction(func){  
    console.log('I am higher order function')  
    func()  
}  
  
higherOrderFunction(callbackFunction);
```

→ this function is HOF as it is taking another function (callbackFunction) as a parameter.

callback function is a function passed as an argument to another function.

Maan lo tumhe DSLR se photo click keri and its a heavy image. Now you want to upload it on facebook.

So, first of all image will be downloaded on facebook server, then it will be compressed, then it will be uploaded on facebook website / profile.

Agar ye poora process from download to upload takes 10 seconds, then 10 seconds tak webpage hang ho jayega. Isko a synchronous karke banaye taki img face book par upload hoti sake and meanwhile mai like, scroll, scroll karta sa hu webpage par.

Webpage hang na ho iske liye upload, compress, download ka process web Api sambhalega.

Maanlo download takes 3 second

① download → 3 seconds

② compress → 2 sec

③ upload → 1 sec

Now I want ki code 1 chale pehle, then

②, then ③. Jabki normally pehle

upload hoga, then compress, then download.

Kyunki download 3 sec baad hoga jabki sabse pehle hona chahiye. We have to

solve this problem.

⇒ Bad practice

```
function downloadFile(){
  console.log("downloading started");

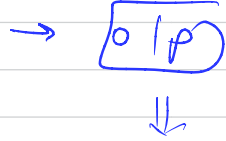
  setTimeout(() => {
    console.log("downloading done");
    compressFile();
  }, 3000);
}

function compressFile(){
  console.log("compressing started");
  setTimeout(() => {
    console.log("compressing done");
    uploadFile();
  }, 2000);
}

function uploadFile(){
  console.log("uploading started");

  setTimeout(() => {
    console.log("uploading done");
  }, 1000);
}

downloadFile();
```



downloading started
downloading done
compressing started
compressing done
uploading started
uploading done

downloaded file ko ~~on~~ baandh diya with
compress file.

compress file ko baandh diya with upload file.

Shreyansh sir ka class wala code:-

```
function downloadFile(callback) {
  const file = 'photo.png'
  console.log('File started to download', file);

  setTimeout(() => {
    callback(file);
  }, 3000);
}

function compressFile(file, callback){
  console.log('File started to compress', file);

  setTimeout(() => {
    const compressedFile = file.split('.')[0] + '.zip';
    callback(compressedFile);
  }, 2000);
}

function uploadFile(file, callback){
  console.log('File started to upload');

  setTimeout(()=>{
    callback('https://facebook.cloud.com/' + file);
  }, 2000)
}

downloadFile(function(file){
  console.log('File downloaded successfully!', file);
  compressFile(file, function(compressedFile){
    console.log('File compressed Successfully!', compressedFile);
    uploadFile(compressedFile, function(finalLink){
      console.log('final link', finalLink);
    });
  });
});
```

O/P ⇒

```
File started to download photo.png
File downloaded successfully! photo.png
File started to compress photo.png
File compressed Successfully! photo.zip
File started to upload
final link https://facebook.cloud.com/photo.zip
```

Explanation of code :-

① Call the downloadFile function:

The downloadFile function is called with a callback function.

After 3 seconds, a message indicating the successful download of the file is logged.

The compressFile function is called with the downloaded file and a callback function.

② Call the compressFile function:

After 2 seconds, a message indicating the successful compression of the file is logged.

The uploadFile function is called with the compressed file and a callback function.

③ Call the uploadFile function:

After 2 seconds, a message indicating the final link of the uploaded file is logged.

Q Let's say, you order a food from 3 websites:-

Zomato	Uberats	Swigato Zpto
Food	Drinks	Sweets
delivery% 3 sec time	1 second	2 seconds

But I want to eat in this sequence → ① Food

② Drink

③ Sweet

```
function getFood(callback){
  setTimeout(() => {
    console.log('Food Arrived!');
    callback();
  }, 3000);
}

function getDrinks(callback){
  setTimeout(() => {
    console.log('Drinks Arrived!');
    callback();
  }, 1000);
}

function getSweets(){
  setTimeout(() => {
    console.log('Sweets Arrived!')
  }, 2000);
}
```

```
getFood(function() {
  getDrinks(function(){
    getSweets();
  })
})
```



O/p



Food Arrived!
Drinks Arrived!
Sweets Arrived!

code gets
too complexed

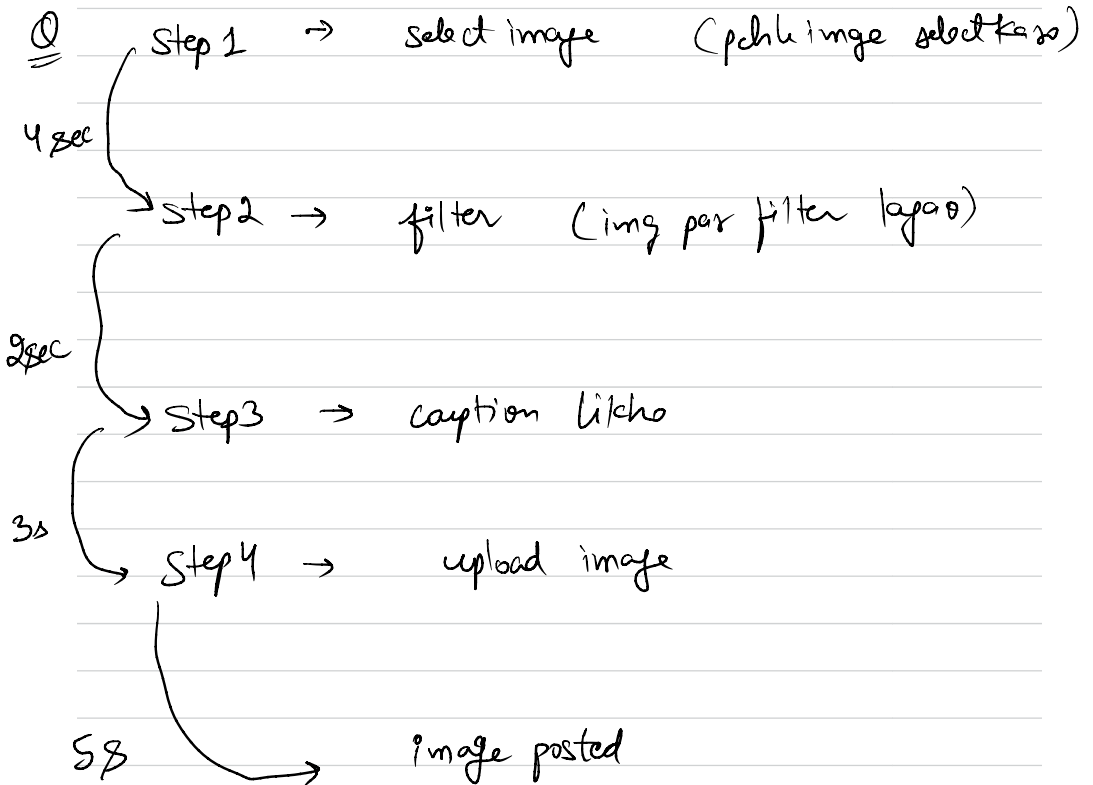


iss part ko callback hell
or pyramid of doom
kehte hai.
code readability kam ho jati hai
nested call back ke case mei. Isliye
ye method jyada use nahi karte.

Code explanation:-

- ① The getFood function is called with a callback function.
- ② After 3 seconds, "Food Arrived!" is printed, and the callback function is executed.
- ③ Inside this callback, the getDrinks function is called with another callback.
- ④ After 1 second, "Drinks Arrived!" is printed, and its callback function is executed.
- ⑤ Inside this callback, the getSweets function is called.
- ⑥ After 2 seconds, "Sweets Arrived!" is printed.

Callback and Call back Hell :-



Ham chahate hai step 2 tabhi ho, jab step 1 complete ho jaaaye. Iske liye step 1 and step 2 ko link krane padega.

```

function step1(fn){
    console.log("selecting image, please wait");    l1

    setTimeout(function(){
        console.log("image has been selected");    l2
        fn("image.jpg");    l3
    }, 4000);
}

function step2(image, fn){
    console.log("applying filter on image");
    setTimeout(function(){
        console.log("filter applied");
        fn("filteredimage.jpg");
    }, 4000);
}

function step3(filter , fn){
    console.log(`Adding caption to the ${filter}`);
    setTimeout(function(){
        console.log('caption done')
        fn('image with caption') //step 4 ka function
    } , 3000)
}

function step4(caption){
    console.log(`uploading your final ${caption}`);
    setTimeout(function(){
        console.log('Image uploaded successfully');
    } , 5000)
}

step1(function(image){
    step2(image, function(filteredimage){
        step3(filteredimage, function(caption){
            step4(caption);
        })
    });
});

```

l3 par hum reverse engineering kar rahi hai,
 Humko ye pata hai ki l2 ke chalne ke baad
 hame step2 function call karna hai. Isliye
 hamne img file ko fn function mei pass
 karke fn ko call kar diya and isi
 fn mei step 2 ko call kara diya.

fn is a wrapper function jisme hamne
Step 2 ko daal diya.

Step 1 (fn)

↪ fn is a call back function.

Step 2 function chalane ke liye fn("image.jpg") l3
sle mei aaya.

⇓

yahi fn hamare

Step 1 function mei as
an argument pass kiya
hai.

Step 1 (function (image) {
}
)

↪ this is fn of l3.

Agar 40 steps hote , then ye code bahut complex
ho jata . Plus code right hand side ki direction mei
Koafi bada ho jata . The code will grow horizontally
kyunki nesting bahut jyada ho jayegi.

The "Pyramid of Doom" refers to a common problem in asynchronous programming, particularly in languages like JavaScript, where you have a series of nested callback functions. As more nested callbacks are added, the code structure visually forms a pyramid shape, which can make the code difficult to read, maintain, and debug. This problem arises because each asynchronous operation is dependent on the completion of the previous one, leading to deeply nested code.

javascript

Copy code

```
doSomething(function(result) {  
  doSomethingElse(result, function(newResult) {  
    doAnotherThing(newResult, function(finalResult) {  
      doFinalThing(finalResult, function() {  
        console.log('All done!');  
      });  
    });  
  });  
});
```

Pyraming of doom is also known as callback hell.

Pros of this method:-

Agar step 1 mei error hai, then step 2 nahi chalega. yahi fayeda hai is method ka.

Disadvantage of this method:-

- ① code grows horizontally
- ② poor readability
- ③ step 2 is dependent on step 1. Agar step 1 mei error hai, then step 2, 3, 4 kabhi call nahi hoge.

Code explanation:-

```
function step1(fn){
    console.log("selecting image, please wait");

    setTimeout(function(){
        console.log("image has been selected");
        fn("image.jpg");
    }, 4000);
}

function step2(image, fn){
    console.log("applying filter on image");
    setTimeout(function(){
        console.log("filter applied");
        fn("filteredimage.jpg");
    }, 4000);
}

function step3(filter , fn){
    console.log(`Adding caption to the ${filter}`);
    setTimeout(function(){
        console.log('caption done')
        fn('image with caption') //step 4 ka function
    } , 3000)
}

function step4(caption){
    console.log(`uploading your final ${caption}`);
    setTimeout(function(){
        console.log('Image uploaded successfully');
    } , 5000)
}

step1(function(image){
    step2(image, function(filteredimage){
        step3(filteredimage, function(caption){
            step4(caption);
        })
    });
});
```

→ code explained on next page:-

Execution Flow:

- **step1:** The first function `step1` is called with a callback function as its argument. This function starts the process by selecting an image.
 - `setTimeout` delays the execution for 4 seconds, after which the `"image has been selected"` message is logged, and the callback function (`fn`) is invoked with the string `"image.jpg"` .
- **step2:** The callback from `step1` calls `step2` , passing `"image.jpg"` and another callback function. `step2` applies a filter to the image.
 - After 4 seconds, the `"filter applied"` message is logged, and the callback function is invoked with `"filteredimage.jpg"` .
- **step3:** The callback from `step2` calls `step3` , passing `"filteredimage.jpg"` and another callback function. `step3` adds a caption to the filtered image.
 - After 3 seconds, the `"caption done"` message is logged, and the callback function is invoked with `"image with caption"` .
- **step4:** The callback from `step3` calls `step4` , passing `"image with caption"` . `step4` uploads the final image with the caption.
 - After 5 seconds, the `"Image uploaded successfully"` message is logged.

Timeline of Execution:

1. **0 seconds:** `step1` starts, logs `"selecting image, please wait"` .
2. **4 seconds:** Logs `"image has been selected"` , then `step2` starts.
3. **8 seconds:** Logs `"filter applied"` , then `step3` starts.
4. **11 seconds:** Logs `"caption done"` , then `step4` starts.
5. **16 seconds:** Logs `"Image uploaded successfully"` .

This sequence ensures that each step waits for the previous one to complete before proceeding, mimicking a real-world process like image processing and uploading, where each step depends on the previous one.

- pending:
- class syntax
 - get set
 - static
 - inheritance
 - Math
 - Promise
 - watch online shreyansh video why nested callback is not preferred.

Pending:-

- Class syntax
- Inheritance
- Static function